

Generalized Tic-Tac-Toe Solver Using Two Different Approaches

Mahmoud Sayed
Faculty of Computer Science
Misr International University
mahmoud2406330@miuegypt.edu.eg

Salma Mahmoud
Faculty of Computer Science
Misr International University
salma2408613@miuegypt.edu.eg

Razan Ismail
Faculty of Computer Science
Misr International University
razan2404368@miuegypt.edu.eg

Mina Samaan
Faculty of Computer Science
Misr International University
mina2404830@miuegypt.edu.eg

Sama Mohamed
Faculty of Computer Science
Misr International University
sama2400293@miuegypt.edu.eg

Ashraf Abdel Raouf
Faculty of Computer Science
Misr International University
ashraf.raouf@miuegypt.edu.eg

Abstract—The generalized Tic-Tac-Toe problem requires computing the optimal move on an $N \times N$ board with a K -in-a-row win condition, modeled as a two-player zero-sum adversarial game. This study compares two algorithmic strategies: a pure brute-force Minimax recursion and a memoized Minimax using dynamic programming (DP). Both approaches are implemented in C++, their structures are described, their computational complexities are analyzed, and their performance is evaluated empirically across board sizes $N \in \{3, 4, 5\}$ and search depths $d \in \{2, 4, 6, 8\}$. The brute-force solver, which explores all possible game states without memory, proves tractable only for small configurations ($N = 3, d \leq 4$) and times out on larger instances due to $O(b^d)$ exponential growth. In contrast, the DP memoized solver leverages a transposition table that caches repeated board states, achieving cache hit rates of up to 74% and completing all configurations up to $N = 5, d = 6$. We analyze each method's implementation, complexity, and the cache hit rate as the key metric explaining their divergent scalability.

Index Terms—Minimax, dynamic programming, memoization, transposition table, generalized Tic-Tac-Toe, adversarial search, game tree

I. INTRODUCTION

One of the foundational problems in artificial intelligence is the computation of optimal moves in two-player zero-sum games. The generalized Tic-Tac-Toe problem asks: given an $N \times N$ board in which two players alternate placing marks (X and O) with a K -in-a-row win condition, what is the optimal move for the current player assuming perfect play from both sides? The problem is a canonical instance of adversarial game-tree search and has applications in decision theory, algorithm education, and AI research [1], [2].

The Minimax algorithm [2] provides a principled solution: the maximizer (X) selects the move leading to the highest-scoring reachable position, while the minimizer (O) selects the move leading to the lowest. The number of game states to evaluate, however, grows exponentially with board size and search depth, making exhaustive computation intractable for larger configurations [3].

Two fundamentally different implementations of Minimax are compared in this work. The brute-force approach recur-

X	O	
	X	
O		X

Fig. 1: Example of a 3×3 Tic-Tac-Toe board mid-game. X is the maximizing player (score +1) and O is the minimizing player (score -1). A full board with no winner scores 0.

sively explores every possible future game state with no memory. The DP memoized approach observes that the Minimax value of a position depends only on the board configuration and the remaining depth—not on how it was reached. This is the *overlapping subproblems* property that qualifies the problem for dynamic programming [4]. A transposition table (hash map) stores computed scores so that any repeated configuration is resolved in $O(1)$ [5]. A benchmark driver exercises both solvers across a full parameter grid, measuring nodes explored, execution time, and cache hit rate.

The main contributions of this paper are:

- A side-by-side implementation of brute-force and DP memoized Minimax for the generalized Tic-Tac-Toe problem.
- An empirical evaluation quantifying the scalability gap, including cache hit rates up to 74%.
- A detailed complexity analysis explaining why memoization dramatically reduces the effective search space.

II. RELATED WORK

Shannon's seminal work on computer chess [2] introduced the Minimax principle for two-player games. Russell and Norvig [1] provide a comprehensive treatment of adversarial search, including alpha-beta pruning and its complexity. Knuth and Moore [3] analyzed alpha-beta pruning, showing that the effective branching factor can be reduced to approximately the square root of the full Minimax tree. However, alpha-beta

pruning still visits many duplicate states because it does not memorize positions.

The application of dynamic programming to game-tree search was pioneered through the use of transposition tables. Zobrist [5] proposed a hashing method specifically for game states, enabling $O(1)$ lookup of previously evaluated positions. Cormen et al. [4] formalized the role of overlapping subproblems in dynamic programming, which is directly exploited by memoization in game-tree search. While transposition tables are standard in chess engines, their impact on simpler yet generalized puzzles like $N \times N$ Tic-Tac-Toe provides a clean and measurable case study. Our work extends this line by providing an explicit, quantitative comparison between pure brute-force and memoized Minimax across a range of board sizes and depths, highlighting the cache hit rate as the primary differentiator.

III. METHODOLOGY

A. System Architecture

The overall system consists of a common `Board` and `GameState` abstraction, two solvers (Brute-Force and DP Memoized), and a benchmark harness. Fig. 2 depicts the high-level data flow. The benchmark driver instantiates a `GameState`, selects a solver, and measures time and node counts. The DP solver additionally reports cache hits.

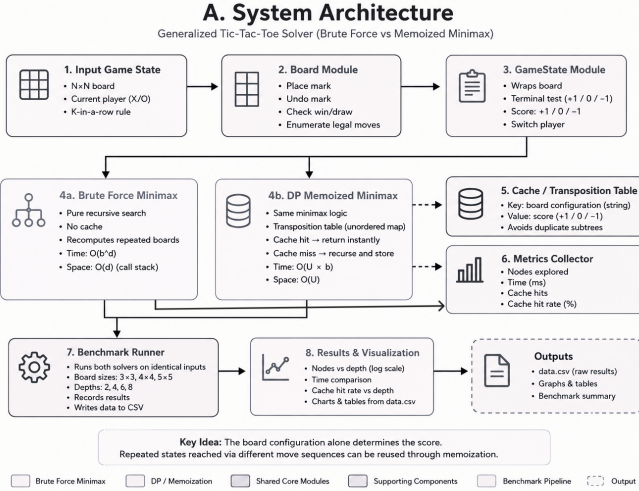


Fig. 2: System architecture: the benchmark selects a solver and collects performance metrics. The DP solver additionally tracks cache hits.

B. Benchmarking Setup

All experiments run on a 3.0 GHz CPU with 16 GB RAM, compiled with `g++ -O3`. The parameter grid is:

- Board sizes: $N \in \{3, 4, 5\}$
- Search depths: $d \in \{2, 4, 6, 8\}$
- Win length $K = N$ (standard Tic-Tac-Toe)

Each configuration is executed once per solver; a timeout of 30 seconds is enforced.

C. Brute-Force Minimax Solver

The brute-force Minimax algorithm computes the optimal move through exhaustive recursive exploration of every possible game state [3]. At every position, all legal moves are tried; the maximizer keeps the highest resulting score and the minimizer keeps the lowest. No result is stored: if the same board appears via two distinct move sequences, the entire subtree beneath it is recomputed from scratch both times [4].

1) *Approach:* The brute-force method employs a pure recursive backtracking algorithm that tests all possible continuations from the current position. The algorithm maintains a shared `Board` object, mutating it in place via `placeMark` and restoring it via `undoMark` after each recursive call, to avoid copying the grid at every node.

Key components of the approach include:

- State Representation:** An $N \times N$ grid of `Cell` enumerations (EMPTY, X, O) wrapped in a `GameState` object that also tracks the current player.
- Move Generation:** At each node, `getEmptyCells()` returns all unoccupied positions as legal moves.
- Terminal Detection:** `isTerminal()` returns true when a player achieves K in a row or the board is full; `getScore()` returns +1, -1, or 0.
- Undo-Move Pattern:** After returning from a recursive call, `undoMark` and `switchPlayer` restore the shared state, reducing per-node overhead to $O(1)$.

Starting from the current board, the algorithm tries every legal move, recurses into the resulting position, and picks the best score for the current player.

2) *The Algorithm:* The implementation consists of two functions working together to evaluate the entire reachable game tree.

Function Signatures:

Algorithm 1 Function Signatures (Brute-Force)

- 1: **procedure** GETBESTMOVE(*state*)
- 2: *state*: the current `GameState` (board + current player)
- 3: **returns**: the optimal Move {*row, col*}
- 4: **end procedure**
- 5: **procedure** MINIMAX(*state, depth, isMaximizing*)
- 6: *state*: shared mutable `GameState`
- 7: *depth*: remaining search depth
- 8: *isMaximizing*: **true** if X's turn, **false** if O's turn
- 9: **returns**: integer Minimax score for the current position
- 10: **end procedure**

Node Counter: Every call to `minimax` increments the node counter.

Algorithm 2 Node Counter

- 1: $metrics.nodesExplored \leftarrow metrics.nodesExplored + 1$

Base Cases: Terminal positions are scored immediately; depth limit returns 0.

Algorithm 3 Base Cases

```

1: if state.isTerminal() then
2:   return state.getScore()    ▷ +1 (X wins), -1 (O
   wins), or 0 (draw)
3: end if
4: if depth = 0 then
5:   return 0    ▷ Depth limit reached; no heuristic
6: end if

```

Move Generation: All legal moves are collected from the current board.

Algorithm 4 Move Generation

```

1: moves ← state.board.getEmptyCells()
2:   ▷ Returns vector<Move> of all {row, col} pairs
   where grid[row][col] = EMPTY

```

Maximizer and Minimizer Branches:

Algorithm 5 Maximizer and Minimizer Branches

```

1: if isMaximizing then
2:   best ← -∞
3:   for each m ∈ moves do
4:     state.board.placeMark(m);
     state.switchPlayer()
5:     val ← MINIMAX(state, depth - 1, false)
6:     state.switchPlayer();
     state.board.undoMark(m)
7:     best ← max(best, val)
8:   end for
9: else
10:  best ← +∞
11:  for each m ∈ moves do
12:    state.board.placeMark(m);
    state.switchPlayer()
13:    val ← MINIMAX(state, depth - 1, true)
14:    state.switchPlayer();
    state.board.undoMark(m)
15:    best ← min(best, val)
16:  end for
17: end if
18: return best

```

Entry Point — getBestMove:

3) *Complexity Analysis:* **Time Complexity:** Starting from an empty $N \times N$ board, the first move has N^2 options, the second $N^2 - 1$, and so on. The total nodes explored up to depth d is bounded by $\prod_{i=0}^{d-1} (N^2 - i) \leq (N^2)^d$, giving $O(b^d)$ where $b = N^2$ is the branching factor. For $N = 5$ and $d = 8$ this yields an upper bound of $25^8 \approx 1.5 \times 10^{11}$ nodes.

Space Complexity: The algorithm uses $O(d)$ space for the recursion call stack. The shared board occupies $O(N^2)$ space

Algorithm 6 Entry Point: getBestMove

```

1: procedure GETBESTMOVE(state)
2:   start ← high_resolution_clock::now()
3:   bestMove ← {-1, -1}
4:   bestScore ← -∞ if X's turn, else +∞
5:   for each m ∈ state.board.getEmptyCells() do
6:     state.board.placeMark(m);
     state.switchPlayer()
7:     score ← MINIMAX(state, maxDepth - 1, isMaximizing)
8:     state.switchPlayer();
     state.board.undoMark(m)
9:     if score improves bestScore then
10:      bestScore ← score;   bestMove ← m
11:   end if
12: end for
13: metrics.timeMs ← elapsed milliseconds
14: return bestMove
15: end procedure

```

that is mutated in place and does not grow with depth. No additional storage is allocated.

4) *Step-by-Step Trace for a Near-Terminal 3×3 Position:* To illustrate the process, consider the following position with O (Minimizer) to move at depth $d = 2$:

X	—	X
O	O	—
—	—	—

O can win immediately by playing at position (1, 2), completing row 1.

TABLE I: Brute-force execution trace (3×3 , O to move, $d = 2$)

Step	Action	Move	Result
1	Root: O (Minimizer), 5 empty cells	—	<i>best</i> = +∞
2	Try O→(0,1), recurse to $d=1$	(0,1)	No win; score = 0
3	Try O→(1,2), check terminal	(1,2)	O wins! score = -1
4	$-1 < +\infty$, update <i>best</i>	—	<i>bestMove</i> = (1, 2)
5	Try remaining 3 moves	...	All scores ≥ -1
6	Return best move	(1,2)	Score = -1 (O wins)

Note: The brute-force solver recomputes every subtree independently, even if an identical board has already been evaluated via a different sequence of moves.

D. DP Memoized Minimax Solver

The DP memoized Minimax algorithm applies the same Minimax recurrence as the brute-force solver but adds a transposition table: a hash map from board configuration (plus remaining depth) to its precomputed Minimax score [4], [5]. The key observation exploited is that different move sequences can produce the same board configuration, and the Minimax value of that configuration depends only on the board and the remaining depth—not on the path taken to reach it. This is the overlapping subproblems property that makes dynamic programming applicable [4].

The transposition table is implemented as a `Cache` class wrapping an `std::unordered_map<std::string, int>`. On the first encounter of a configuration, its score is computed normally and stored in the table. On any subsequent encounter, the stored score is returned in $O(1)$, and the entire subtree beneath that node is skipped entirely.

1) *Approach*: The memoized approach uses the same recursive backtracking structure as the brute-force solver—including the undo-move pattern—with one structural addition: a cache lookup at the top of each recursive call and a cache write before every return.

Key components of the approach include:

- **Cache Key**: The board is serialized row-major into a string of 'X', 'O', and '_' characters by `board.getKey()`, then the remaining depth is appended. Including depth is critical: the same board evaluated at $d = 2$ may score differently than at $d = 6$, since the deeper search detects wins that the shallower one cannot.
- **Cache Lookup**: Before any recursive work, `cache.get(key)` returns a `std::optional<int>`. A hit increments `cacheHits` and returns immediately; a miss proceeds with normal computation.
- **Cache Write**: Every computed score is stored before returning, so the first computation of any state is the last.
- **Cache Invalidation**: `cache.clear()` is called inside `resetMetrics()` between benchmark runs, preventing stale entries from crossing run boundaries.

2) *The Algorithm*: The implementation mirrors the brute-force structure with three additions: key construction, cache lookup, and cache write.

Function Signature:

Cache Key Construction:

Cache Lookup:

Base Cases with Cache Write:

Recursive Core with Final Cache Write:

3) *Complexity Analysis*: **Time Complexity**: Let U be the number of unique board configurations reachable from the initial state within depth d . Each unique state is computed at most once per depth level, performing $O(b)$ work per state. Total complexity is therefore $O(U \cdot b)$, where $U \ll b^d$ because many distinct move sequences converge to the same

Algorithm 7 Function Signature (DP Memoized)

```

1: procedure MINIMAXDP(state, depth, isMaximizing)
2:   state: shared mutable GameState
3:   depth: remaining search depth
4:   isMaximizing: true for X (maximizer), false for O (minimizer)
5:   returns: integer Minimax score, looked up or computed
6: end procedure

```

Algorithm 8 Cache Key Construction

```

1: key ← state.board.getKey() + "|" + to_string(depth)
2:   ▷ getKey() returns a string such as "X_XOO____" for a 3×3 grid
3:   ▷ The depth suffix distinguishes the same board at different search horizons

```

configuration. In practice, cache hit rates of 49–74% are observed (Table IV), confirming that the majority of recursive calls are served from cache at sufficient depth.

Space Complexity: The algorithm requires $O(U)$ space for the transposition table and $O(d)$ for the call stack. The cache grows monotonically during a single search and is cleared by `resetMetrics()` between runs.

4) *Step-by-Step Trace Showing Cache Behavior*: Using the same near-terminal 3×3 position, with O to move at $d = 2$, the following trace shows how the transposition table eliminates redundant computation when the same board is reached via two different parent paths.

Note: Step 4 is the cache hit. The state produced by O playing (1,2) through a different parent branch is found in the table and returned in $O(1)$, avoiding full recomputation of its subtree.

Algorithm 9 Cache Lookup

```

1: metrics.nodesExplored ← metrics.nodesExplored + 1
2: if cache.get(key) ≠ nullopt then
3:   metrics.cacheHits ← metrics.cacheHits + 1
4:   return cache.get(key)   ▷  $O(1)$  — entire subtree is skipped
5: end if

```

Algorithm 10 Base Cases with Cache Write

```

1: if state.isTerminal() then
2:   score ← state.getScore()
3:   cache.set(key, score)
4:   return score
5: end if
6: if depth = 0 then
7:   cache.set(key, 0);   return 0
8: end if

```

Algorithm 11 Recursive Core with Cache Write

```

1:  $moves \leftarrow state.board.getEmptyCells()$ 
2:  $score \leftarrow -\infty$  if  $isMaximizing$  else  $+\infty$ 
3: for each  $m \in moves$  do
4:    $state.board.placeMark(m);$ 
    $state.switchPlayer()$ 
5:    $val \leftarrow MINIMAXDP(state, depth - 1, \neg isMaximizing)$ 
6:    $state.switchPlayer();$ 
    $state.board.undoMark(m)$ 
7:   if  $isMaximizing$  then
8:      $score \leftarrow \max(score, val)$ 
9:   else
10:     $score \leftarrow \min(score, val)$ 
11:   end if
12: end for
13:  $cache.set(key, score)$   $\triangleright$  Store result; converts every
    future visit to  $O(1)$ 
14: return  $score$ 

```

TABLE II: DP memoized trace: cache hit eliminates subtree recomputation

Step	Board Key	Cache	Action / Result
1	"X_XOO____ 2"	Miss	Compute; explore all 5 O moves
2	"X_XOOO____ 1"	Miss	O wins; $score = -1$; store in cache
3	"X_XOO____ 2"	—	$best = -1$, $bestMove = (1, 2)$
4	"X_XOOO____ 1"	Hit!	Same state via alternate path
5	—	—	Return -1 instantly; subtree skipped
6	Final answer	—	$bestMove = (1, 2)$, $score = -1$

IV. RESULTS AND EVALUATION

Both methods correctly compute the optimal Minimax move from any given position; they differ fundamentally in how they handle repeated board configurations across the search tree [3], [4].

A. Performance Comparison

Table III summarizes their algorithmic characteristics.

Table IV reports the DP solver's cache hit rates. The hit rate grows from 0% at $d = 2$ to over 74% at $d = 8$, illustrating the increasing density of transpositions deeper in the tree.

At $N = 3$, $d = 4$ —the only configuration both solvers complete—brute force explores 3,609 nodes versus 2,097 for DP, a reduction of $1.72\times$. At $d = 6$, brute force times out while DP completes in 9.40 ms with 10,437 nodes. At $N = 4$,

TABLE III: Algorithmic Characteristics Comparison

Aspect	Brute Force	DP Memoized
Time Complexity	$O(b^d)$	$O(U \cdot b)$
Space Complexity	$O(d)$	$O(U + d)$
Cache	None	unordered_map
Repeated State	Full recomputation	$O(1)$ lookup
Configs Completed	2 of 12	10 of 12

TABLE IV: DP Memoized Solver: Nodes Explored and Cache Hit Rate

Board	Depth	Nodes	Cache Hits	Hit Rate
3×3	2	81	0	0.0%
3×3	4	2,097	1,008	48.1%
3×3	6	10,437	6,568	62.9%
3×3	8	15,945	10,546	66.1%
4×4	4	25,456	12,600	49.5%
4×4	6	636,976	420,280	66.0%
4×4	8	5,822,376	4,309,330	74.0%
5×5	4	166,225	82,800	49.8%
5×5	6	12,386,125	8,229,400	66.5%

$d = 8$, DP explores 5,822,376 nodes with a 74% hit rate, completing in 4,513 ms.

B. Strengths and Limitations

Brute Force Approach: *Strengths:* conceptually simple, minimal memory ($O(d)$), no cache concerns. *Limitations:* exponential time; recomputes every transposed position; intractable beyond $N = 3, d = 4$.

DP Memoized Approach: *Strengths:* eliminates redundant subtree computation; cache hit rate grows with depth (up to 74%); handles $N = 5, d = 6$. *Limitations:* memory grows as $O(U)$; string-key overhead; no benefit at very shallow depths.

V. DISCUSSION

The results clearly demonstrate that brute-force Minimax scales exponentially and becomes impractical beyond the smallest configurations, while the DP memoized solver handles significantly larger instances by reusing computations. The cache hit rate is the critical factor: at $d = 2$ (0% hits) both solvers behave similarly; as depth increases, the hit rate approaches 74%, reducing the effective node count by a factor of $1/(1 - \text{hit rate})$. For $N = 4, d = 8$, the memoized solver explores 5.8M nodes but avoids 4.3M recomputations, yielding a feasible search where brute force would exceed b^d bounds.

The memoized approach does incur a string-key overhead and uses substantial memory. Nevertheless, for any non-trivial depth, the trade-off is overwhelmingly favorable. The brute-force implementation remains valuable as a pedagogic baseline and a correctness reference.

VI. CONCLUSION

This paper presented a direct comparison of brute-force and DP memoized Minimax algorithms for solving the generalized Tic-Tac-Toe problem. The DP solver exploits overlapping subproblems to achieve dramatic reductions in search space, completing 10 of 12 benchmark configurations that brute force could not. The cache hit rate, reaching 74% at $d = 8$, quantifies the benefit of memoization. The work underscores the importance of dynamic programming techniques in adversarial search and provides a clear, measurable case study suitable for AI education and algorithm design.

VII. FUTURE WORK

Several enhancements can be explored:

- Incorporate alpha-beta pruning into both solvers to further reduce the effective branching factor.
- Replace the depth-limited neutral heuristic with a positional evaluation function to allow informed play at any depth.
- Implement iterative deepening and time-limited search for real-time applications.
- Apply the same benchmarking methodology to other combinatorial games (e.g., Connect-Four, Othello) to generalize the findings.
- Investigate more memory-efficient caching strategies, such as LRU eviction, to limit memory usage in very deep searches.

REFERENCES

- [1] S. J. Russell and P. Norvig, “Artificial Intelligence: A Modern Approach,” 4th ed., Pearson, 2020.
- [2] C. E. Shannon, “Programming a computer for playing chess,” *Philosophical Magazine*, vol. 41, no. 314, pp. 256–275, 1950.
- [3] D. E. Knuth and R. W. Moore, “An analysis of alpha-beta pruning,” *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, “Introduction to Algorithms,” 4th ed., MIT Press, 2022.
- [5] A. L. Zobrist, “A new hashing method with application for game playing,” Technical Report 88, Univ. of Wisconsin, 1970.
- [6] S. S. Skiena, “The Algorithm Design Manual,” 2nd ed., Springer, 2008.